

Enterprise Google Cloud Data Platform Blueprint

VOLUME 13

Terraform and Infrastructure as Code

Enterprise IaC Architecture, Golden Modules, State Governance and Implementation Blueprint
Architecture Baseline 1.0 | 12 July 2026

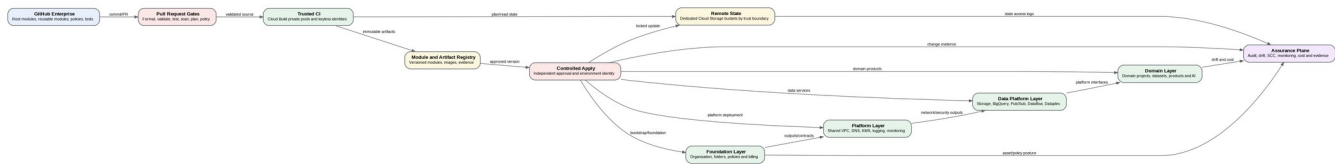


Figure 0-1. Enterprise Terraform architecture from source and policy gates through secured state, layered roots and assurance.

Series status. This is the final missing volume in the 14-volume blueprint. Completion enables Phase 3 cross-volume architecture assurance. It does not itself perform Phase 3.

Document Control

Field	Value
Document	Volume 13 - Terraform and Infrastructure as Code
Version	1.0
Status	Detailed reference architecture and implementation blueprint
Date	12 July 2026
Owner	Cloud Foundation and Platform Engineering
Approvers	Enterprise Architecture, Security, Network, Data Platform, DevSecOps, SRE, BCDR and FinOps
Scope	Organization through domain infrastructure, modules, state, policy, testing, migration, drift and recovery
Inherited volumes	01-12 and 14; Volume 09 defines source/build/release governance
Next phase	Phase 3 cross-volume architecture assurance

Table of Contents

1. Executive Summary
2. Inherited Context and Assumptions
3. Infrastructure-as-Code Strategy
4. Terraform Architecture
5. Operating Model and Ownership
6. Repository Design
7. Module Design
8. Root Modules
9. Reusable Modules
10. Environment Configuration
11. Variable Strategy
12. State Management
13. Remote State
14. State Locking
15. State Security
16. Provider Management
17. Version Pinning
18. Organization Deployment
19. Folder Deployment
20. Project Factory
21. Shared VPC Deployment
22. IAM Deployment
23. KMS Deployment
24. VPC Service Controls Deployment
25. Logging Deployment
26. Monitoring Deployment
27. Cloud Storage Deployment
28. BigQuery Deployment
29. Pub/Sub Deployment
30. Dataflow Deployment
31. Dataplex Deployment
32. Composer Deployment
33. Vertex AI Deployment
34. Artifact Registry Deployment
35. Cloud Build Deployment
36. Policy as Code
37. Terraform Testing
38. Static Analysis
39. Security Scanning
40. Plan Review
41. Approval
42. Apply
43. Drift Detection
44. Import Strategy
45. Migration Strategy
46. Rollback
47. Disaster Recovery
48. Module Documentation
49. Module Versioning

50. Module Publishing

51. Golden Modules

52. Landing-Zone Modules

53. Domain Modules

54. Infrastructure Manager Assessment

55. Service Assessments

56. ADRs

57. Risks

58. Roadmap

59. Validation

60. Production Readiness

61. Cross-Volume Dependencies

62. Decisions and Next Actions

63. References

1. Executive Summary

Volume 13 makes infrastructure delivery repeatable, secure and recoverable across the enterprise Google Cloud data platform. Terraform is the authoritative desired-state engine. Golden modules encode the landing-zone, Zero Trust, data, AI, SRE, DR and FinOps controls defined in previous volumes. Root modules remain small and independently owned so that state, approval, recovery and lifecycle boundaries match business risk.

Mandatory baseline. All production infrastructure is represented in version-controlled Terraform or a documented time-bound exception. Remote state, keyless automation, policy gates, independent production approval, drift detection and tested recovery are mandatory.		
Outcome	Architecture response	Measure
Repeatability	Versioned golden modules and standardized roots	Managed coverage >=95%
Security	WIF, scoped identities, state protection and policy gates	Zero long-lived CI keys
Reliability	Small state boundaries, locks and recovery exercises	State recovery within tier RTO/RPO
Governance	Plan evidence, approvals and post-apply validation	100% production changes traceable
Agility	Project/domain factories and self-service catalog	Onboarding lead time
Cost	Reusable modules and label/budget controls	Allocation coverage and avoided rework

2. Inherited Context and Assumptions

The design inherits the Fortune 100 scale, AMER/EMEA/APAC data planes, dev/int/test/staging/production/sandbox environments, Public/Internal/Confidential/Restricted/Regulated classifications, Shared VPC, group-based IAM, private networking, VPC Service Controls, selective CMEK, data products and Tier 0-2 recovery objectives.

Mandatory baseline. The master prompt remains authoritative. Volume 9 controls source, build and promotion; Volume 8 controls Zero Trust and supply chain; Volumes 10-12 control cost, operations and recovery.	
Inherited decision	Terraform implementation
Central platform + federated domains	Platform owns golden modules; domains own approved root configurations
Group-based IAM	Modules reject direct user grants except break-glass workflows
Private-by-default	Shared VPC, Private Google Access, PSC and firewall modules
Regional data planes	Separate roots/state for AMER, EMEA and APAC
Data-product FinOps	Labels, tags, budgets and cost center variables required
Tiered recovery	State/source/config recovery matched to Tier 0/1/2

3. Infrastructure-as-Code Strategy

IaC is a product capability, not a collection of scripts. The platform team provides paved roads, module support, policy libraries, pipelines, state services, documentation and upgrade programs. Domain teams consume these capabilities and remain accountable for their resource lifecycle, data classification, SLO and cost.

Principle	Decision
Desired state	Git is the reviewed source of desired configuration; state maps it to real resources
Layering	Primitive modules compose into golden modules; roots deploy lifecycles
Managed-first	Use provider resources and managed services; scripts only for gaps
Policy before privilege	Plans pass policy and risk review before privileged apply
Recoverability	Source, config, providers, modules and state are independently recoverable
No hidden ownership	Every root, module, state prefix and resource has an owner

4. Terraform Architecture

The architecture separates authoring, validation, state, apply and assurance. This separation limits blast radius and enables independent duties.

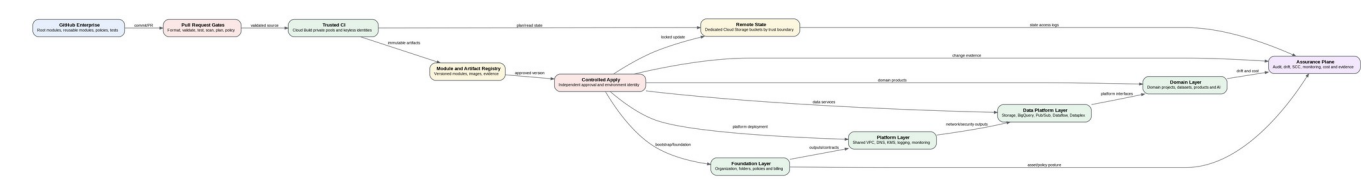


Figure 4-1. Enterprise Terraform architecture with layered deployment and assurance planes.

Plane	Components	Responsibility
Source	GitHub repositories, CODEOWNERS, tags	Versioned desired state
Validation	Cloud Build/GitHub Actions, tests, scanners, policy	Prove safety and compliance
State	Cloud Storage backends and locks	Resource mapping and concurrency
Apply	Scoped service accounts and approvals	Authorized mutation
Assurance	Audit, Asset, SCC, drift, monitoring and cost	Detect divergence and impact

5. Operating Model and Ownership

A central Cloud Foundation team owns organization and foundational modules. Network, Security, Data Platform and AI Platform teams own their service modules. Domain teams own root configurations for approved products. DevSecOps owns pipeline standards; SRE owns recovery and operational readiness.

Artifact	Accountable	Responsible	Consulted
Module interface	Module product owner	Platform engineering	Security, consumers
Root configuration	Workload/platform owner	Delivery team	Module owner, FinOps
State backend	Cloud Foundation/Security	Platform SRE	BCDR
Policy library	Cloud Governance/Security	Policy engineering	Architecture
Production apply	Resource owner	CI apply identity	Independent approver
Provider upgrade	Terraform platform owner	Module owners	Consumers

6. Repository Design

Repositories separate reusable module products from deployable roots while preserving discoverability and cross-cutting policy. A federated multi-repository implementation may mirror the logical structure where ownership and release cadence differ.

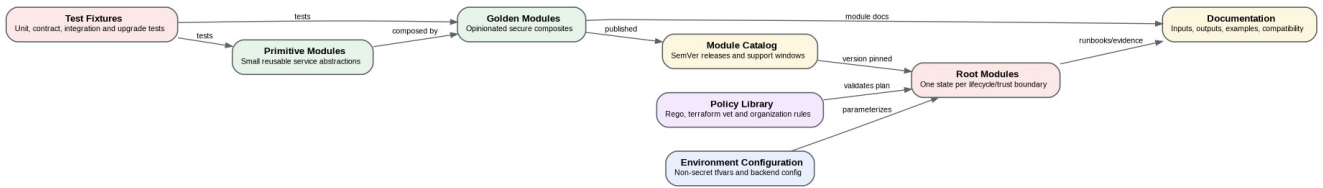


Figure 6-1. Logical repository and module layers, independent of physical monorepo or multi-repository implementation.

Recommended logical repository structure.

```

modules/
  primitive/
  golden/
roots/
  organization/
  platform-network/
  platform-security/
  data-platform/
  domain-product/
environments/{dev,int,tst,stg,prd}/
policies/ tests/ pipelines/ scripts/ docs/

```

- Do not commit state, plan binaries, credentials or generated provider caches.
- Commit dependency lock files for root modules and test compatible platforms.
- Use CODEOWNERS matching real operational ownership.
- Keep environment values non-secret and schema validated.

7. Module Design

Modules expose stable, documented interfaces and encode mandatory controls without hiding important architecture. Primitive modules wrap a narrow service abstraction. Golden modules compose resources into an approved enterprise capability.

Design rule	Requirement
Cohesion	One capability and lifecycle
Inputs	Typed, validated and minimal
Outputs	Stable resource interfaces, not sensitive values
Defaults	Secure and operable; avoid surprising resource creation
Dependencies	Explicit module/provider constraints
Lifecycle	Prevent destroy only for truly protected resources and document recovery
Examples	Runnable non-production examples
Tests	Unit, integration, upgrade and policy coverage

8. Root Modules

Root modules are deployable units with a backend, provider configuration, pinned module versions, environment inputs and runbook. They are partitioned by lifecycle, trust boundary, owner and recovery requirement.

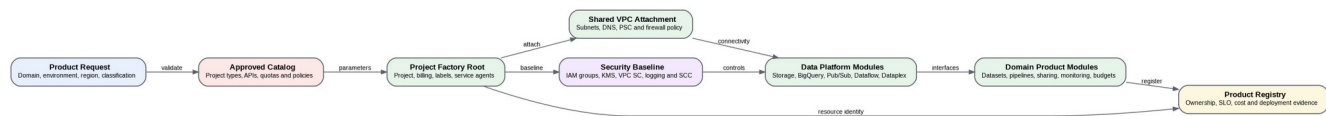


Figure 8-1. Project factory and domain deployment sequence from governed request to registered product.

Do	Avoid
One state per bounded root	Single organization-wide state
Explicit upstream outputs/contracts	Deep remote-state coupling
Independent apply identity	One enterprise owner service account
Named owner and tier	Orphaned roots
Plan-safe migration path	Refactoring without moved/import blocks

9. Reusable Modules

Reusable modules reduce duplication only when they have product ownership, versioning, tests, documentation and support. Copy-paste modules are not reuse.

Class	Purpose	Example
Primitive	Small provider resource abstraction	KMS key ring/keys
Golden	Opinionated enterprise capability	Project factory, Shared VPC, data platform
Composition	Root-level combination for a product	Domain data product
Policy module	Standard organization policy/control set	Restricted workload baseline

10. Environment Configuration

Environment configuration is separated from reusable logic. Production and non-production use different projects, identities, backends and approvals. Configuration files contain identifiers and non-secret values only.

Configuration	Pattern
Backend	Separate partial backend HCL per root/environment
Variables	Reviewed tfvars or structured YAML converted deterministically
Secrets	Runtime retrieval; avoid storing in Terraform state when possible
Cross-root values	Published interface registry or narrowly readable state outputs
Feature differences	Explicit variables or separate composition, not hidden workspace branching

11. Variable Strategy

Variables are typed contracts. Validation prevents invalid environments, locations, classifications, labels and resource names before provider calls.

Variable class	Storage	Example	Control
Non-secret environment	Version control	region,environment,domain	Review and schema validation
Resource identity	Version control/generated catalog	project_id,dataset_id	Naming policy and uniqueness check
Sensitive secret	Secret Manager/runtime	database password	Never tfvars; short-lived access
Sensitive configuration	Encrypted CI variable/Secret Manager	external token	Redacted plan/log handling
Cross-root output	Remote state contract or registry	network URI,KMS key ID	Read-only access and versioned interface
Derived value	locals/data source	labels,service agent	Deterministic and documented

Typed variable validation pattern.

```
variable "environment" {
  type = string
  validation {
    condition = contains(["dev","int","tst","stg","prd","sbx"], var.environment)
    error_message = "Use an approved environment code."
  }
}
```

12. State Management

Terraform state is sensitive operational data. It can contain identifiers, metadata and provider-returned values. State is isolated by root and trust boundary, accessed only by automation and time-bound recovery roles, and never distributed by email or collaboration tools.

State decision	Standard
Granularity	Per bounded root/lifecycle
Location	Dedicated Cloud Storage backend project/bucket
Naming	Stable prefix derived from scope/environment/region
Access	Plan read; apply read/write; recovery time-bound
Protection	Versioning, retention, public prevention, audit and optional CMEK
Operations	Lock timeout, serial lineage, backup/recovery evidence

13. Remote State

Cloud Storage is the default backend. Bootstrap state infrastructure is created separately before migrating local bootstrap state to the secured remote backend. Backend configuration uses partial configuration; credentials are supplied by keyless runtime identity.

Partial GCS backend configuration.

```
terraform { backend "gcs" {} }

# init
terraform init -backend-config=environments/production/backend.hcl
```

Implementation note. Cloud Storage backend behavior, locking, authentication and encryption options must be validated against the current Terraform backend documentation before rollout.

14. State Locking

Backend-supported state locking is mandatory for write operations. Pipelines use a bounded lock timeout and serialize applies per root. Disabling locks is prohibited outside an approved state-recovery incident.

- Use queue/concurrency controls in CI in addition to backend locking.
- Do not force-unlock without verifying the lock owner and active operation.
- Record forced unlocks as incidents or controlled operational events.
- Never use `-lock=false` to bypass a slow pipeline.

15. State Security

State security uses dedicated projects, bucket-level IAM, public access prevention, uniform access, audit logs, version history, retention, threat detection and recovery procedures. State KMS administration is separated from state object administration where CMEK is required.

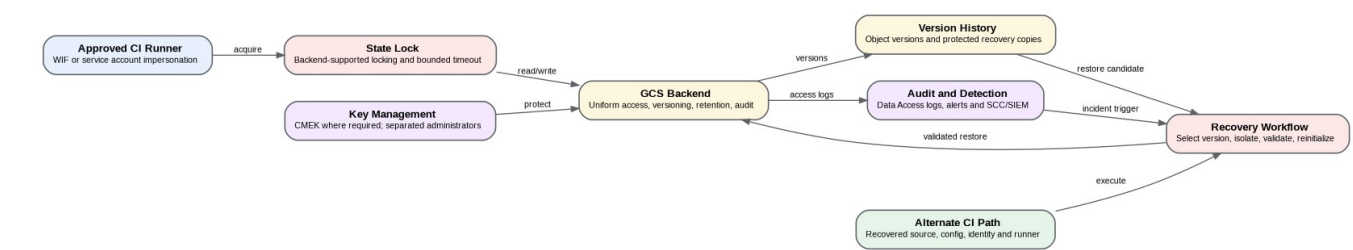


Figure 15-1. State protection, access, versioning, audit and controlled restoration.

State class	Bucket/project	Prefix	Access	Protection	Recovery	RTO/RPO
Bootstrap/ Foundation	Dedicated restricted project	bootstrap/*	Two-person privileged group	Versioning, retention, CMEK, prevent destroy	Documented isolated restore	Tier0 <1h/<5m
Network/Security	Dedicated platform state project	platform/ {network security}/*	Service-specific apply groups	Versioning, retention, audit alerts	Restore generation + refresh-only plan	Tier0 <1h/<5m
Data platform	Regional platform state project	data-platform/ {region}/{env}	Platform CI identities	Versioning, retention, CMEK by policy	Alternate CI + restore	Tier1 <4h/<15m
Domain	Regional/domain partitioned prefix	domains/ {domain}/{env}/ {region}	Domain apply identity + platform break- glass	Versioning, lifecycle, audit	Restore and source recovery	Tier1/2

16. Provider Management

Root modules declare required providers. Providers are configured at roots and passed to child modules; child modules declare requirements but do not embed credentials. The google provider is default; google-beta is explicit and limited to resources requiring it.

Control	Requirement
Source	hashicorp/google and approved providers only
Alias	Use aliases for explicit project/region contexts
Credentials	Application default credentials/WIF; no service-account key files
Upgrade	Automated candidate PR, tests and staged roots
Beta	Document feature need and exit criteria
Lock file	Commit and verify checksums for supported runner platforms

17. Version Pinning

Terraform core, providers and modules use constraints that permit controlled compatible upgrades while preventing unreviewed major changes. Production releases record exact resolved versions and the plan digest.

Version constraint and pinned module release example.

```
terraform {  
  required_version = ">= 1.8, < 2.0"  
  required_providers {  
    google = { source = "hashicorp/google", version = "~> 6.0" }  
  }  
}  
module "project" { source = "git::ssh://.../project-factory.git?ref=v3.4.1" }
```

18. Organization Deployment

Organization roots manage high-impact controls: folders, organization policies, billing integration, central groups and foundational service identities. They use a dedicated state and apply identity with dual approval for destructive or privilege-expanding changes.

Aspect	Standard
Scope	Organization only
Pipeline	Dedicated foundation lane
Approval	Cloud Foundation + Security
Validation	Hierarchy, policy and audit export
Rollback	Forward-fix or controlled policy restore

19. Folder Deployment

Folder modules encode legal entity, environment, region and platform/domain boundaries. Folder moves are treated as high-risk because inherited policy, IAM and billing visibility can change.

Aspect	Standard
Pattern	Stable hierarchy with explicit parent IDs
Guardrail	No ad hoc folder creation
Test	Inherited policies and IAM
Migration	Inventory children and simulate impact

20. Project Factory

The project factory creates standardized projects, billing attachment, labels, APIs, group IAM, network attachment, logging, monitoring and registration. It is the principal self-service boundary for platform and domain teams.

Aspect	Standard
Mandatory	Disable default network
Metadata	environment, domain, cost center, owner, classification
Identity	Groups and dedicated runtime service accounts
Lifecycle	Create, suspend, archive and close procedures

21. Shared VPC Deployment

Shared VPC roots are regional/platform lifecycles managed by the Network Platform team. Service project attachments, subnet CIDRs, secondary ranges, routes, DNS, NAT, PSC and firewall policies are reviewed together.

Aspect	Standard
Control	IPAM allocation before plan
Security	Private Google Access and flow logs
Change	CIDR/route changes require impact review
Test	Positive and negative connectivity

22. IAM Deployment

IAM modules use groups and scoped service accounts. Additive member resources are preferred when ownership is shared. Authoritative bindings are allowed only when the root owns the complete role membership.

Aspect	Standard
Avoid	Direct users and primitive roles
Conditions	Time/resource/context where applicable
Separation	IAM admin distinct from workload deployer
Validation	Access analyzer/negative persona tests

23. KMS Deployment

KMS roots create regional key rings, keys, rotation and role separation. Protected keys use lifecycle safeguards and a documented destroy/retirement process. Service agents receive only required encrypter/decrypter access.

Aspect	Standard
Location	Aligned with protected service/data
Roles	Key admin separate from crypto user
Rotation	Policy driven and monitored
Recovery	Key availability included in DR tests

24. VPC Service Controls Deployment

VPC Service Controls are managed centrally with dry-run first, explicit restricted services, access levels and reviewed ingress/egress. Project membership changes are coordinated with project factory and service enablement.

Aspect	Standard
Stage	Dry run then enforced
Evidence	Policy denied logs and exfiltration tests
Ownership	Security Platform
Exception	Time-bound reviewed perimeter bridge/egress

25. Logging Deployment

Organization/folder/project sinks, log buckets, exclusions, retention and writer identities are Terraform managed. Routing is designed before workload onboarding because missing historical logs cannot be recreated.

Aspect	Standard
Scope	Central audit/security plus workload operations
Cost	Filter noisy low-value logs
Security	Protect sinks and archives
Test	Synthetic event reaches destination

26. Monitoring Deployment

Monitoring modules create notification channels, dashboards, SLOs and alert policies with owner, severity and runbook labels. Workload modules emit resources; observability modules bind standard alerts without circular dependencies.

Aspect	Standard
Standard	Actionable symptoms over raw causes
Ownership	Product/platform on-call
Promotion	Same definitions, environment thresholds
Test	Synthetic alert and routing

27. Cloud Storage Deployment

Cloud Storage modules enforce approved location, uniform access, public prevention, encryption, retention/lifecycle, labels and logging where required. Destruction defaults to false for production data.

Aspect	Standard
Data classes	Landing, quarantine, curated, archive, state
Security	CMEK by classification and VPC SC
Lifecycle	Explicit retention and privacy/legal controls
Test	Access and lifecycle policy

28. BigQuery Deployment

BigQuery modules create datasets, tables/views where infrastructure-owned, policy tags/masking bindings, reservation assignments where applicable, CMEK and access interfaces. Business SQL remains in Dataform/analytics release lanes.

Aspect	Standard
Boundary	Terraform for infrastructure and stable access; SQL tools for transformations
Location	Regional/multi-region consistency
IAM	Dataset/product groups
Safety	No delete_contents_on_destroy in production

29. Pub/Sub Deployment

Pub/Sub modules manage topics, schemas, subscriptions, retention, retry, dead-letter, CMEK, labels and service-agent IAM. Producer and consumer ownership are explicit.

Aspect	Standard
Contract	Topic/schema version
Recovery	Retention and snapshot/replay pattern
Security	Publisher/subscriber least privilege
Operations	Backlog and oldest-message alerts

30. Dataflow Deployment

Terraform creates supporting identities, networks, KMS, templates/triggers and selected managed jobs. Streaming update/drain/replay is coordinated with Volume 4 and Volume 9; Terraform must not blindly recreate a stateful job.

Aspect	Standard
Artifact	Immutable Flex Template metadata
Identity	Dedicated worker SA
Network	Private workers/subnet
Lifecycle	on_delete drain and explicit update strategy

31. Dataplex Deployment

Dataplex/Knowledge Catalog infrastructure modules establish lakes/zones/assets or current catalog resources where applicable, discovery configuration, IAM and metadata integration. Product terminology and provider resource support are validated during implementation.

Aspect	Standard
Ownership	Governance platform + domains
Metadata	Classification, lineage and product registration
Policy	Regional and data perimeter alignment
Validation	Asset discovery and permissions

32. Composer Deployment

Managed Airflow environments are Terraform managed with pinned supported image, dedicated identity, private networking, KMS and environment segmentation. DAG releases remain in the Volume 9 pipeline.

Aspect	Standard
Root	Environment infrastructure only
Artifact	DAGs/plugins deployed separately
Upgrade	Image compatibility test and snapshot
DR	Configuration/source/snapshot recovery

33. Vertex AI Deployment

Terraform provisions AI projects, APIs, service accounts, artifact buckets, network/perimeter and approved platform resources. Models, prompts, pipelines and evaluations follow Volume 7/9 release lanes.

Aspect	Standard
Boundary	Foundation infrastructure vs ML assets
Security	Dedicated identities, KMS and data perimeter
Region	Model/service availability validated
Operations	Quotas, monitoring and cost labels

34. Artifact Registry Deployment

Artifact Registry modules create regional repositories, cleanup policies, CMEK and producer/consumer IAM. Repositories are immutable systems of record for pipeline, module-support and application artifacts.

Aspect	Standard
Format	Separate repositories by artifact/trust
Security	Scanning and write/read separation
Retention	Policy with protected releases
Evidence	Digest referenced in release manifest

35. Cloud Build Deployment

Cloud Build modules manage connections/triggers, private pools, service accounts, network access, logs and approvals/integration. Build configuration remains versioned source governed by Volume 9.

Aspect	Standard
Identity	Trigger-specific SA; WIF for external source
Network	Private pools for restricted access
Separation	Plan and apply triggers/identities
Audit	Build logs, substitutions and plan digest

36. Policy as Code

Policy as code is layered. HCL validation catches syntax and type defects; lint and security scanners catch common configuration risks; gcloud terraform vet and OPA enforce Google Cloud and enterprise rules; post-apply checks verify real policy, asset and security posture. No single policy tool is treated as complete assurance.

Gate	Tool/pattern	Blocks	Evidence
Formatting	terraform fmt	Non-canonical HCL	CI log
Initialization/validation	terraform init -backend=false; validate	Syntax/provider/module errors	CI log
Lint	TFLint	Provider and style violations	SARIF/log
Security scan	Checkov/tfsec or enterprise scanner	High/critical misconfiguration	Scan result
Google policy	gcloud beta terraform vet	Organization policy/control violations	Policy report
Enterprise policy	OPA/Rego	Naming, labels, architecture rules	Decision log
Plan review	Saved plan + JSON + risk classifier	Unexpected destroy/replace or privilege expansion	Approved plan digest
Post-apply	Asset/policy/smoke checks	Control regression	Validation report

Plan policy validation sequence.

```
terraform plan -out=tfplan
terraform show -json tfplan > tfplan.json
gcloud beta terraform vet tfplan.json --policy-library=policies/google
conftest test tfplan.json --policy policies/opa
```

Current capability note. gcloud beta terraform vet is documented as a policy validation capability and has resource coverage limitations. Use layered controls and validate its current support before relying on it.

37. Terraform Testing

Testing covers modules and roots, not only syntax. Tests prove secure defaults, outputs, updates, upgrades, destruction behavior and integration with real Google Cloud services in disposable projects.

Stage	Tests	Environment	Pass criteria
Module unit	terraform test with mocks	CI	Assertions pass
Static	fmt, validate, TFLint, docs	CI	Zero blocking findings
Security/policy	Scanner, vet, OPA	CI	No unapproved high/critical or policy deny
Integration	Create/update/destroy fixture	Ephemeral project	Expected outputs and clean destroy
Upgrade	Old supported version -> candidate	Ephemeral project	No unexpected replacement
Plan contract	Consumer roots	Nonprod state	No breaking output/input change
Production validation	Smoke, asset, monitoring, cost labels	Production	All mandatory controls pass
Recovery	State/source/config recovery	Isolated exercise	Within tier RTO/RPO

38. Static Analysis

Every pull request runs formatting, initialization without production backend where appropriate, validation, TFLint, documentation checks and custom repository rules. Generated documentation must match code.

- Treat warnings as tracked debt; production blocking thresholds are defined by policy.
- Run static analysis on every module and changed consumer root.
- Verify lock files and approved provider sources.
- Reject hard-coded project numbers, credentials and unrestricted CIDRs unless explicitly approved.

39. Security Scanning

Security scanning inspects HCL, modules, provider dependencies, containers used by pipelines and generated plans. Findings are risk-ranked and tied to exceptions with owner and expiry.

Scan	Target	Blocking baseline
IaC misconfiguration	HCL and plan	High/critical unapproved
Secrets	Repository/history	Any verified secret
Dependencies	CI tools/modules/images	Critical exploitable
Policy	Plan and live asset	Any mandatory deny
IAM	Bindings and conditions	Direct user/broad privilege
Network	Routes/firewalls/egress	Unapproved exposure

40. Plan Review

The saved plan is the review artifact. Reviewers evaluate create/update/delete/replace actions, IAM and network expansion, stateful resource impact, provider changes, unknown values, cost drivers and downstream dependencies.

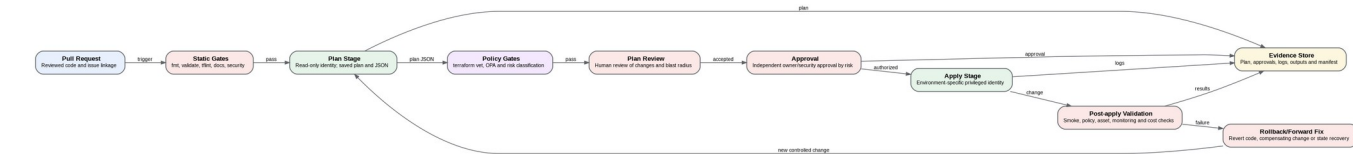


Figure 40-1. Controlled plan, policy, approval, apply, validation and recovery workflow.

Risk signal	Review requirement
Destroy/replace stateful resource	Service owner + BCDR
Privilege expansion	Security approval
Network/perimeter egress	Network + Security
Organization policy/folder move	Cloud Foundation + Security
Large cost/capacity change	FinOps + owner
No-op	Automated evidence still retained

41. Approval

Production approval is independent from the author and apply identity. Approval requirements are risk-based but cannot be bypassed by labeling a change urgent. Emergency changes use break-glass, incident command and post-change reconciliation.

Change	Minimum approval
Routine nonprod	Root owner
Routine production	Root owner independent reviewer
Security/network/IAM expansion	Root owner + specialist owner
Tier 0 destructive/replace	Executive/service owner + BCDR/SRE
Emergency	Incident commander + authorized break-glass approver

42. Apply

Apply consumes the exact reviewed plan when feasible, under a scoped environment identity. The pipeline records plan digest, source commit, module/provider versions, approvers, apply log, outputs and post-apply result.

- Never run interactive production apply from a workstation.
- Use bounded lock timeout and pipeline concurrency per root.
- Stop on unexpected plan change; do not auto-replan after approval without renewed review.
- Run smoke, policy, asset, monitoring and cost-label checks after apply.

43. Drift Detection

Scheduled refresh-only plans and Cloud Asset/security signals identify divergence. Drift is classified before remediation because provider-computed values, emergency changes, malicious changes and unmanaged resources need different responses.

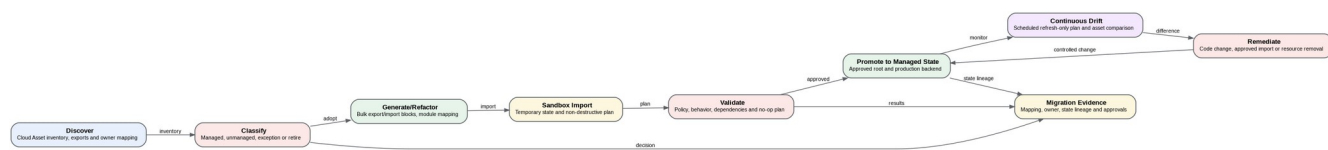


Figure 43-1. Existing-resource adoption and ongoing drift lifecycle.

Drift type	Detection	Default action	Escalation
Benign console change	Refresh-only plan	Revert through code or approved import	Resource owner
Security weakening	Policy/SCC/plan	Immediate containment and incident	Security on-call
Provider-computed difference	Scheduled plan noise	Suppress only with documented lifecycle rule	Module owner
Emergency change	Change record + drift	Codify after incident; preserve evidence	Incident commander
Unknown unmanaged resource	Asset comparison	Classify owner; adopt or remove	Cloud governance
State/resource mismatch	Plan/read errors	Freeze applies and use recovery runbook	Platform SRE

44. Import Strategy

Import is state adoption, not automatic architecture compliance. Existing resources are inventoried, assigned owners, represented using approved modules, imported first into isolated state, and validated for a no-destructive-change plan.

Declarative import block pattern where supported by the Terraform version and provider resource.

```
import {  
  to = module.data_platform.google_storage_bucket.landing  
  id = "egcdp-sales-prd-landing"  
}
```


45. Migration Strategy

Migration from manual, Deployment Manager, scripts or other Terraform layouts proceeds in waves. Bulk export can accelerate discovery but generated code is refactored, secured, tested and documented before production adoption.

Wave	Scope	Activities	Exit
0	Mobilize	Inventory, owners, state strategy, repository and policies	Approved target model
1	Bootstrap	State project/buckets, CI identities, module catalog	Recoverable secured backends
2	Foundation	Organization, folders, projects, network, KMS, logging	No-op plans and policy evidence
3	Data platform	Storage, BigQuery, Pub/Sub, Dataflow, Dataplex, Composer	Regional platform roots operational
4	Domains	Domain project/product roots and imports	Priority domains managed
5	Optimize	Drift, upgrades, module lifecycle and retirement	>=95% managed coverage

46. Rollback

Terraform rollback is a reviewed new desired-state change. Reverting code can still produce destructive actions, so the new plan is reviewed. Stateful data recovery uses service-native mechanisms. State operations repair mapping, not business data.

Prohibited anti-pattern. Do not restore an old state file merely to make Terraform show a desired plan. State must reflect live resource identity. Use the recovery runbook and refresh-only validation.

47. Disaster Recovery

DR covers source repositories, module artifacts, provider access, backend configuration, CI identity, runners and state. Tier 0/1 roots have tested alternate execution and state-version restoration procedures aligned to Volume 12.

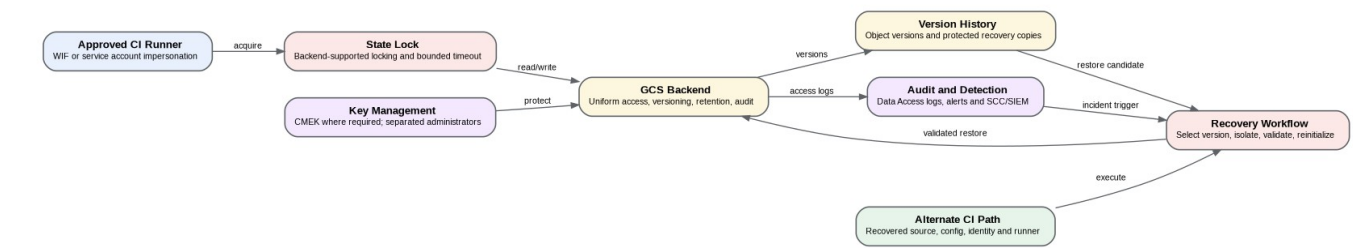


Figure 47-1. State and automation recovery controls reused for Terraform disaster recovery.

Asset	Recovery
Source	Protected GitHub repository/mirror and signed tags
Modules	Versioned source/catalog and release archive
Provider binaries	Reinitialize from approved source/cache; verify lock checksums
State	Cloud Storage generation restore under dual approval
Backend config	Recovered from protected source/config repository
CI	Alternate runner/project and WIF/service-account path
Secrets	Secret Manager recovery/rotation; not source files

48. Module Documentation

Each module includes purpose, architecture, usage, inputs, outputs, required providers, IAM, network, encryption, state impact, examples, tests, upgrade path, known limitations, ownership and support window.

Document	Requirement
README	Human architecture and quick start
Generated inputs/outputs	Synchronized with code
Examples	Runnable and tested
CHANGELOG	User-visible changes and migration notes
SECURITY	Security model and reporting
SUPPORT	Owners, versions and escalation
ADR link	Why major choices were made

49. Module Versioning

Modules use semantic versioning. Patch releases fix compatible defects; minor releases add compatible capabilities; major releases change interfaces or behavior and require migration guidance. Pre-release versions cannot be used in production without exception.

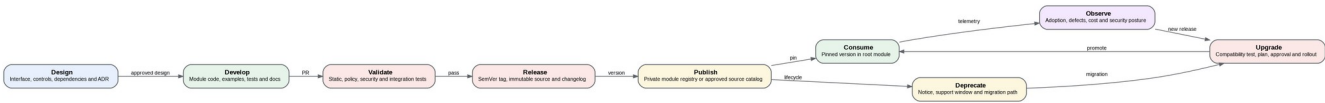


Figure 49-1. Golden module design, validation, signed release, publication, consumption, upgrade and retirement lifecycle.

50. Module Publishing

Approved modules are published from protected release workflows using signed tags and immutable source references. The catalog records owner, version, support status, compatibility, documentation, tests and consumers.

- Publishing requires all tests and security/policy gates.
- The release is built once; consumers pin the exact version.
- Deprecated versions remain available through the support window.
- Critical security fixes trigger coordinated consumer upgrade campaigns.

51. Golden Modules

Golden modules encode mandatory enterprise controls and reduce consumer choice to safe parameters. They are not black boxes: generated resources, IAM and operational behavior remain documented and inspectable.

Module	Type	Owner	Version policy	Inputs	Outputs	Support
project_factory	Golden	Cloud Foundation	SemVer; major approval	folder,billing,labels,APIs,IAM	project ID/number	N,N-1 majors
shared_vpc	Golden	Network Platform	SemVer	host,subnets,service projects	network/subnet URIs	N,N-1
kms	Primitive/Golden	Security Platform	SemVer	location,keys,members	key IDs	N,N-1
vpc_service_controls	Golden	Security Platform	SemVer	policy,projects,services,levels	perimeter name	N,N-1
data_platform	Golden	Data Platform	SemVer	project,location,KMS,names,labels	resource IDs	N,N-1
dataflow_template	Primitive	Data Engineering	SemVer	template,identity,network,KMS	job ID	N,N-1
composer	Golden	Data Platform	SemVer	image,network,identity,KMS	environment ID	N,N-1
vertex_ai_foundation	Golden	AI Platform	SemVer	project,location,KMS	SA,bucket	N,N-1

52. Landing-Zone Modules

Landing-zone modules implement organization, folder, project factory, Shared VPC, IAM, KMS, VPC Service Controls, logging, monitoring, security and CI/CD foundations from Volume 2 and Volume 8. Their state and approval are isolated from workload roots.

Module group	Examples
Organization	folders, policies, billing and tags
Network	Shared VPC, subnet, DNS, NAT, PSC and firewall
Security	KMS, IAM, VPC SC, SCC and audit
Operations	logging, monitoring and alert channels
Delivery	Artifact Registry, Cloud Build pools/triggers and WIF

53. Domain Modules

Domain modules create projects, datasets, buckets, topics, pipelines, monitoring, budgets and data-product registration inside inherited guardrails. Domain autonomy is configuration and lifecycle ownership, not permission to bypass foundation controls.

Domain input	Example
Identity	domain owner/steward/operator/consumer groups
Placement	AMER/EMEA/APAC region and data location
Classification	Public through Regulated
Product	name, version, SLO and cost center
Services	approved APIs and quotas
Controls	KMS/perimeter/access/retention profile

54. Infrastructure Manager Assessment

Infrastructure Manager is a managed Google Cloud service that deploys Terraform configurations and supports previews/deployments. It is evaluated for bounded Google Cloud deployments and managed execution. The enterprise baseline remains repository-led Terraform with Volume 9 controls; adoption requires validation of supported providers, state/deployment behavior, IAM, regions, quotas, cost and recovery.

Use it when	Do not select it when
A managed Google Cloud execution plane reduces runner operations	Cross-cloud/provider set or enterprise workflow is unsupported
Preview/deployment resource model fits the lifecycle	Existing Cloud Build pipeline controls/evidence cannot be reproduced
Bounded blueprint ownership and IAM are clear	State/control-plane integration is ambiguous
Current service capabilities pass security and DR review	Selection is only to avoid module/pipeline engineering

Architecture status. Optional capability. Run a controlled non-production pilot before making it an enterprise default.

55. Structured Service Assessments

Selected services and technologies directly support state, execution, security, policy, migration and assurance. Each assessment uses the 32-field master template. Current quotas, regions, preview status, IAM and encryption support must be validated during implementation.

55.1 Terraform on Google Cloud

Assessment field	Enterprise assessment
Service name	Terraform on Google Cloud
Purpose	Declarative provisioning and lifecycle management
Enterprise architecture role	Authoritative IaC engine
Business capabilities supported	Repeatable foundation, platform and domain delivery
Technical capabilities	Providers, modules, plan/apply, state, import and tests
Benefits	Broad Google Cloud coverage and ecosystem
Limitations	Provider/API semantics and state expertise required
Service quotas	Provider and service quotas apply
Regional availability considerations	Runner and target resource locations matter
High-availability characteristics	Control plane depends on backend/CI design
Disaster-recovery considerations	Recover source, providers, config and state
Security model	Keyless credentials and scoped provider identities
IAM requirements	Plan/read and apply roles separated
Network requirements	Runner needs Google API/private network access
VPC Service Controls considerations	Validate each managed service and pipeline path
Encryption capabilities	Backend/target service encryption
CMEK support	Target services; backend CMEK where configured
Logging	CLI/provider and CI logs
Monitoring	Pipeline and drift metrics
Alerting	Plan/apply/drift failures
Cost drivers	CI minutes, engineering and target resources
Cost-optimization techniques	Reusable modules and efficient tests
Operational responsibilities	Terraform platform + resource owners
Integration points	Git, Cloud Build, Storage, Google APIs
Alternatives	Pulumi, Config Connector, scripts, Infrastructure Manager
Selection criteria	Enterprise provider coverage and team standard
Best practices	Small roots, pinned versions, reviewed plans
Reference architecture	Figures 4-1 and 40-1
Sample configuration	implementation/modules and roots
Common pitfalls	Monolithic state and secrets in variables
Anti-patterns	Workstation production apply
Production-readiness checklist	State, identity, policy, tests, recovery and ownership ready

55.2 Cloud Storage state backend

Assessment field	Enterprise assessment
Service name	Cloud Storage state backend
Purpose	Remote Terraform state storage
Enterprise architecture role	Protected state system of record
Business capabilities supported	Concurrent auditable infrastructure operations
Technical capabilities	GCS backend, object versions, retention and IAM
Benefits	Managed durable storage and integration
Limitations	State remains sensitive; backend behavior must be understood
Service quotas	Bucket/object/IAM quotas
Regional availability considerations	Choose compliant location; state need not colocate all resources but governance applies
High-availability characteristics	Managed storage availability by location class
Disaster-recovery considerations	Restore object generations and alternate access path
Security model	Uniform bucket access, public prevention, scoped IAM
IAM requirements	Object read/write only by plan/apply/recovery roles
Network requirements	Google API/private access
VPC Service Controls considerations	Can be placed in service perimeter subject to validation
Encryption capabilities	Encryption at rest/in transit
CMEK support	CMEK via bucket default where required
Logging	Data Access and admin logs
Monitoring	Bucket access/errors/version growth
Alerting	Unauthorized access and failed operations
Cost drivers	Storage, operations, logging and KMS
Cost-optimization techniques	Small state, lifecycle and avoid duplication
Operational responsibilities	Cloud Foundation/Security/SRE
Integration points	Terraform backend, KMS, Logging
Alternatives	Terraform Cloud/Enterprise or other backend
Selection criteria	Google Cloud-centric estate and control needs
Best practices	Dedicated project/bucket, versioning, retention, recovery test
Reference architecture	Figures 15-1 and 47-1
Sample configuration	implementation/bootstrap-state
Common pitfalls	Broad object admin and missing versioning
Anti-patterns	Shared general-purpose bucket
Production-readiness checklist	Access, protection, logs, restore and RTO/RPO proven

55.3 Cloud Build

Assessment field	Enterprise assessment
Service name	Cloud Build
Purpose	Trusted Terraform validation and apply execution
Enterprise architecture role	Primary Google-hosted CI execution from Volume 9
Business capabilities supported	Automated plans, policy and applies
Technical capabilities	Triggers, private pools, service accounts and logs
Benefits	Managed scaling and Google integration
Limitations	Pipeline design and identity remain enterprise responsibilities
Service quotas	Build/concurrency/pool quotas
Regional availability considerations	Pools and target connectivity by region
High-availability characteristics	Managed service; private pool capacity design
Disaster-recovery considerations	Alternate pool/project/source path
Security model	Trigger SA, worker pool and WIF controls
IAM requirements	Scope-specific service accounts
Network requirements	Private pools/VPC for restricted targets
VPC Service Controls considerations	Validate build and target service paths
Encryption capabilities	Managed encryption and supported CMEK options
CMEK support	Validate log/storage/pool support
Logging	Build and audit logs
Monitoring	Build duration/failure/queue
Alerting	Failed/unauthorized apply alerts
Cost drivers	Build minutes, pools, network and logs
Cost-optimization techniques	Right-size pools and cache safely
Operational responsibilities	DevSecOps + root owners
Integration points	GitHub, Artifact Registry, Storage, APIs
Alternatives	GitHub Actions, Infrastructure Manager, Terraform Cloud
Selection criteria	Trusted Google execution and network needs
Best practices	Separate plan/apply, immutable images, no secrets
Reference architecture	Figure 40-1
Sample configuration	implementation/pipelines/cloudbuild-terraform.yaml
Common pitfalls	Default service account overprivilege
Anti-patterns	Single trigger can apply everything
Production-readiness checklist	Identity, network, logs, approvals and alternate execution tested

55.4 Artifact Registry

Assessment field	Enterprise assessment
Service name	Artifact Registry
Purpose	Store immutable CI/tool/container artifacts
Enterprise architecture role	Artifact system of record
Business capabilities supported	Trusted reproducible Terraform toolchains and pipeline artifacts
Technical capabilities	Repositories, IAM, scanning, cleanup and regional storage
Benefits	Digest-based releases and Google integration
Limitations	Not a Terraform module registry in every workflow; choose catalog design
Service quotas	Repository/storage/API quotas
Regional availability considerations	Regional placement and replication strategy
High-availability characteristics	Managed regional service
Disaster-recovery considerations	Replicate/rebuild from signed source
Security model	Producer/consumer IAM and vulnerability controls
IAM requirements	Writers separate from readers
Network requirements	Private Google access/PSC patterns as supported
VPC Service Controls considerations	Supported service; validate perimeter
Encryption capabilities	Encryption at rest/in transit
CMEK support	CMEK supported for repositories subject to constraints
Logging	Audit logs
Monitoring	Storage, downloads and scanning
Alerting	Vulnerability/unauthorized write alerts
Cost drivers	Storage, egress, scanning
Cost-optimization techniques	Cleanup while preserving releases
Operational responsibilities	DevSecOps
Integration points	Cloud Build, modules/tool images
Alternatives	GitHub packages/private registry
Selection criteria	Immutable artifact and scanning needs
Best practices	Use digests, protected releases, retention
Reference architecture	Figure 7-1
Sample configuration	Terraform repository resource
Common pitfalls	Mutable latest tags
Anti-patterns	Treating source artifact as unversioned cache
Production-readiness checklist	Repository/IAM/KMS/scanning/retention/recovery ready

55.5 Cloud KMS

Assessment field	Enterprise assessment
Service name	Cloud KMS
Purpose	Protect state and provisioned service data
Enterprise architecture role	Enterprise key control plane
Business capabilities supported	Regulated encryption and separation of duties
Technical capabilities	Key rings, keys, rotation, IAM and logs
Benefits	Managed key lifecycle and service integration
Limitations	Location/service support and key availability are dependencies
Service quotas	Key/rate quotas
Regional availability considerations	Key location must align with service
High-availability characteristics	Managed regional/multi-regional characteristics
Disaster-recovery considerations	Key availability and restore/retirement planning
Security model	Separate key admins and crypto users
IAM requirements	Scoped key IAM and service agents
Network requirements	Google API/private access
VPC Service Controls considerations	Supported service and perimeter considerations
Encryption capabilities	Managed HSM/software/external options
CMEK support	It is the CMEK service
Logging	Admin and data access logs
Monitoring	Request/error/rotation metrics
Alerting	Denied access and rotation alerts
Cost drivers	Keys, operations, HSM/EKM
Cost-optimization techniques	Use only where required and consolidate responsibly
Operational responsibilities	Security Platform
Integration points	Storage, BigQuery, Pub/Sub, Dataflow, Composer, Vertex AI
Alternatives	Google-managed encryption, HSM, EKM
Selection criteria	Classification/regulatory requirement
Best practices	Prevent destroy, rotate and test service agents
Reference architecture	Figures 15-1 and 47-1
Sample configuration	implementation/modules/kms
Common pitfalls	Destroying key before data lifecycle ends
Anti-patterns	Terraform apply identity as KMS admin
Production-readiness checklist	Location/IAM/rotation/DR and dependent-service tests complete

55.6 Cloud Asset Inventory and resource export/import tooling

Assessment field	Enterprise assessment
Service name	Cloud Asset Inventory and resource export/import tooling
Purpose	Discover assets and support adoption/drift assurance
Enterprise architecture role	Migration and assurance data source
Business capabilities supported	Inventory, ownership, export and compliance comparison
Technical capabilities	Asset search/export and gcloud resource-config tooling
Benefits	Broad discovery and migration acceleration
Limitations	Exported Terraform is generated starting material, not approved architecture
Service quotas	API/export quotas and resource coverage
Regional availability considerations	Organization/project scope; outputs stored appropriately
High-availability characteristics	Managed inventory service
Disaster-recovery considerations	Re-run inventory/export; retain migration evidence
Security model	Read-only inventory identities and protected outputs
IAM requirements	Asset viewer/export permissions
Network requirements	Google API access
VPC Service Controls considerations	Validate perimeter/output destinations
Encryption capabilities	Managed service/output storage encryption
CMEK support	Output destination dependent
Logging	Audit logs
Monitoring	Export/job status and coverage
Alerting	Failed exports or unexpected assets
Cost drivers	Queries, storage and processing
Cost-optimization techniques	Scope exports and retain only required evidence
Operational responsibilities	Cloud Governance/Migration
Integration points	Terraform import, BigQuery, Storage
Alternatives	Manual inventory, third-party CMDB
Selection criteria	Existing estate adoption and drift
Best practices	Assign owner, refactor, sandbox import
Reference architecture	Figure 43-1
Sample configuration	implementation/scripts/bulk_export.sh
Common pitfalls	Applying generated code directly
Anti-patterns	Assuming export covers all semantics
Production-readiness checklist	Coverage, permissions, output security and no-destructive plan validated

55.7 Infrastructure Manager

Assessment field	Enterprise assessment
Service name	Infrastructure Manager
Purpose	Managed deployment of Terraform configurations
Enterprise architecture role	Optional managed IaC execution plane
Business capabilities supported	Preview and deploy bounded Google Cloud blueprints
Technical capabilities	Deployments, previews and maintained providers
Benefits	Reduced runner/toolchain operations
Limitations	Provider/workflow/region/state behavior may not fit all enterprise roots
Service quotas	Service deployment quotas
Regional availability considerations	Service locations and target resource regions
High-availability characteristics	Managed service
Disaster-recovery considerations	Recover source/config and recreate deployment per documented model
Security model	Deployment SA and service agent controls
IAM requirements	Scoped deployment service account
Network requirements	Target API/network access
VPC Service Controls considerations	Validate service and target paths
Encryption capabilities	Managed encryption; target service controls
CMEK support	Validate service-specific support
Logging	Deployment/audit logs
Monitoring	Deployment state and errors
Alerting	Failed preview/deployment alerts
Cost drivers	Managed deployment and underlying build/resource costs
Cost-optimization techniques	Use for qualified bounded deployments
Operational responsibilities	Cloud Foundation/DevSecOps
Integration points	Terraform, Cloud Build, Storage, APIs
Alternatives	Cloud Build pipeline, Terraform Cloud, GitHub Actions
Selection criteria	Managed execution fits requirements
Best practices	Pilot, preview, scoped SA and evidence integration
Reference architecture	Section 54
Sample configuration	implementation/infrastructure_manager
Common pitfalls	Assuming feature parity with bespoke pipelines
Anti-patterns	Replacing governance with a service call
Production-readiness checklist	Provider/IAM/state/DR/evidence/cost pilot approved

55.8 Cloud Logging and Audit Logs

Assessment field	Enterprise assessment
Service name	Cloud Logging and Audit Logs
Purpose	Record Terraform, state and infrastructure changes
Enterprise architecture role	Audit and operational evidence plane
Business capabilities supported	Change traceability, investigation and compliance
Technical capabilities	Admin/Data Access logs, sinks, buckets and alerts
Benefits	Central Google Cloud audit integration
Limitations	Data Access volume/cost and payload sensitivity
Service quotas	Logging quotas
Regional availability considerations	Regional bucket/retention design
High-availability characteristics	Managed service
Disaster-recovery considerations	Central/archive recovery design
Security model	IAM, sinks, retention and access controls
IAM requirements	Log writers/viewers separated
Network requirements	Google API/private paths
VPC Service Controls considerations	Supported with perimeter design
Encryption capabilities	Encryption at rest/in transit
CMEK support	CMEK for log buckets where supported
Logging	It is the logging service
Monitoring	Ingestion, exclusions and routing metrics
Alerting	Missing audit/change and access anomalies
Cost drivers	Ingestion, retention, routing and analytics
Cost-optimization techniques	Filter noise without losing evidence
Operational responsibilities	SRE/Security
Integration points	Cloud Build, Storage, SCC, SIEM
Alternatives	Third-party logging
Selection criteria	Enterprise audit requirement
Best practices	Enable state/object and privileged Data Access logs
Reference architecture	Assurance plane in Figure 4-1
Sample configuration	implementation/modules/central_logging
Common pitfalls	Sensitive state content in logs
Anti-patterns	Disabling logs to save cost
Production-readiness checklist	Required logs, routing, access, retention and alert tests pass

55.9 Security Command Center

Assessment field	Enterprise assessment
Service name	Security Command Center
Purpose	Central security posture and findings
Enterprise architecture role	Post-apply security assurance
Business capabilities supported	Detect misconfiguration, threats and compliance drift
Technical capabilities	Findings, posture and integrations
Benefits	Independent signal beyond plan-time policy
Limitations	Edition/detector coverage and timing vary
Service quotas	Finding/API quotas
Regional availability considerations	Organization and service availability
High-availability characteristics	Managed service
Disaster-recovery considerations	Export findings/evidence and recovery integration
Security model	Organization-level roles and finding access
IAM requirements	Security admin/viewer separation
Network requirements	Google API access
VPC Service Controls considerations	Service perimeter/integration validation
Encryption capabilities	Managed encryption
CMEK support	Not generally a customer data CMEK use case; validate
Logging	Audit/finding exports
Monitoring	Finding counts/severity
Alerting	Critical finding routing
Cost drivers	Edition, exports and SIEM
Cost-optimization techniques	Tune without suppressing critical controls
Operational responsibilities	Security Operations
Integration points	Asset, Logging, SIEM, Terraform remediation
Alternatives	Third-party CSPM
Selection criteria	Independent cloud posture need
Best practices	Treat findings as input to controlled code change
Reference architecture	Assurance plane Figure 4-1
Sample configuration	Post-apply validation integration
Common pitfalls	Auto-remediating without context
Anti-patterns	Using scan as substitute for policy/design
Production-readiness checklist	Coverage, routing, ownership and remediation runbooks ready

56. Architecture Decision Records

Twelve Volume 13 ADRs formalize authority, repository/module design, state, identity, policy, drift, migration, rollback, release and managed deployment decisions.

ADR	Decision	Status
ADR-1301	Use Terraform as the authoritative Infrastructure-as-Code engine	Approved reference
ADR-1302	Use layered repositories with reusable modules and deployable roots	Approved reference
ADR-1303	Partition root modules and state by lifecycle and trust boundary	Approved reference
ADR-1304	Use versioned golden modules as the paved road	Approved reference
ADR-1305	Use Cloud Storage remote state with isolated secured backends	Approved reference
ADR-1306	Separate Terraform plan and apply identities and approvals	Approved reference
ADR-1307	Enforce layered policy-as-code before and after apply	Approved reference
ADR-1308	Use scheduled refresh-only plans for drift detection	Approved reference
ADR-1309	Adopt existing resources through controlled export, refactor and import	Approved reference
ADR-1310	Treat rollback as a new controlled change and use service-native recovery	Approved reference
ADR-1311	Use semantic versioning and signed module releases	Approved reference
ADR-1312	Use Infrastructure Manager selectively for managed Terraform deployments	Approved reference

ADR-1301 - Use Terraform as the authoritative Infrastructure-as-Code engine

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Terraform is the default authoritative engine for Google Cloud foundation, platform and domain infrastructure. Console and ad hoc scripts are exceptions reconciled to code.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1302 - Use layered repositories with reusable modules and deployable roots

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Separate versioned primitive/golden modules from root modules, environment configuration, policy and tests.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1303 - Partition root modules and state by lifecycle and trust boundary

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Use small roots for organization, network, security, data platform and domain lifecycles rather than one monolithic state.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1304 - Use versioned golden modules as the paved road

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Publish opinionated secure modules with SemVer, compatibility tests, support windows and deprecation paths.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1305 - Use Cloud Storage remote state with isolated secured backends

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Store state in dedicated buckets/projects with uniform access, versioning, retention, auditing and CMEK where required.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1306 - Separate Terraform plan and apply identities and approvals

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Use read-oriented plan identities and scoped privileged apply identities with independent production approval.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1307 - Enforce layered policy-as-code before and after apply

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Combine Terraform validation, lint/security tools, gcloud terraform vet/OPA and post-apply asset/security checks.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1308 - Use scheduled refresh-only plans for drift detection

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Detect drift continuously and classify emergency, benign, security and state mismatch differences.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1309 - Adopt existing resources through controlled export, refactor and import

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Bulk export is discovery input; production adoption requires approved modules, sandbox state and no-destructive-change validation.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1310 - Treat rollback as a new controlled change and use service-native recovery

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Prefer configuration revert/forward-fix and service-native restore. State manipulation is limited to mapping recovery under incident control.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1311 - Use semantic versioning and signed module releases

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Consumers pin module versions; major changes require migration guidance and staged upgrade validation.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

ADR-1312 - Use Infrastructure Manager selectively for managed Terraform deployments

Field	Decision record
Status / Date	Approved reference / 12 July 2026
Problem	Enterprise IaC must be secure, modular, recoverable and auditable.
Context	Global regulated data platform with federated teams and high change volume.
Drivers	Security, state integrity, blast radius, recovery, policy, productivity and cost.
Alternatives	Manual operations; scripts; monolith; alternate engine; managed service only.
Decision	Infrastructure Manager may be used for bounded Google Cloud deployments where its managed previews/deployments fit; it does not replace the enterprise repository, policy, state and Volume 9 governance model.
Rationale	Creates a paved road with explicit lifecycle, state and approval boundaries.
Benefits	Repeatability, lower drift, faster onboarding and evidence.
Tradeoffs	Module investment, migration and provider lifecycle management.
Risks	Mis-scoped state, upgrade defects, privilege or incomplete import.
Mitigations	Small roots, pinned versions, tests, policy, scoped identity and recovery.
Implementation	Update modules, roots, state, pipelines, policies and runbooks.
Cost	Track CI, storage, logging and platform engineering investment.
Security	Keyless access, least privilege and sensitive state protection.
Operations	Owner, drift, upgrade, support and recovery duties required.
Review	12 October 2026 or material trigger

57. Risks and Mitigations

The risk register is reviewed quarterly and after incidents, provider changes or large migrations. High residual risks require named executive acceptance and expiry.

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-1301	State compromise exposes sensitive values or enables destructive control	Medium	Critical	Dedicated projects, least privilege, audit, CMEK where required, no secrets in state where avoidable	Security Platform
R-1302	Oversized root creates excessive blast radius	High	High	Lifecycle/trust-boundary roots and dependency contracts	Cloud Foundation
R-1303	Provider upgrade causes unexpected changes	Medium	High	Lock files, version constraints, upgrade tests and staged rollout	Module Owners
R-1304	Manual console changes create drift	High	Medium	Deny/guardrails, scheduled drift, exception workflow	Cloud Governance
R-1305	Import recreates or destroys production resources	Medium	Critical	Sandbox import, module refactor, reviewed no-op plan	Migration Lead
R-1306	Authoritative IAM resources overwrite other owners	Medium	Critical	Ownership model, additive members by default, IAM tests	Security Architecture
R-1307	Module breaking change impacts many domains	Medium	High	SemVer, support window, upgrade tests and canary consumers	Platform Engineering
R-1308	State lock bypass corrupts state	Low	Critical	Never disable locking; incident recovery for lock issues	Platform SRE
R-1309	CI identity is overprivileged	Medium	Critical	Plan/apply split, scope-specific SAs, WIF and periodic review	DevSecOps
R-1310	Terraform attempts rollback of stateful data incorrectly	Medium	Critical	Service-native recovery and forward-fix; state ops only for mapping	BCDR/SRE
R-1311	Beta resource/provider semantics change	Medium	Medium	Explicit google-beta use, validation and review triggers	Architecture
R-1312	Policy tool preview/beta limitations create false assurance	Medium	High	Layered policies, direct tests, SCC/asset post-apply checks	Security Platform

58. Implementation Roadmap

The roadmap establishes state and module foundations before broad migration. Representative Tier 0/1 roots prove recovery and policy before domain scale.

Wave	Scope	Activities	Exit
0	Mobilize	Inventory, owners, state strategy, repository and policies	Approved target model
1	Bootstrap	State project/buckets, CI identities, module catalog	Recoverable secured backends
2	Foundation	Organization, folders, projects, network, KMS, logging	No-op plans and policy evidence
3	Data platform	Storage, BigQuery, Pub/Sub, Dataflow, Dataplex, Composer	Regional platform roots operational
4	Domains	Domain project/product roots and imports	Priority domains managed
5	Optimize	Drift, upgrades, module lifecycle and retirement	>=95% managed coverage

- Pilot organization read-only planning, one regional platform root and one domain product root.
- Exercise state recovery before production onboarding.
- Prioritize unmanaged high-risk resources and frequently changed foundations.
- Track managed coverage, drift age, module adoption, upgrade latency and failed-change rate.

59. Validation and Acceptance Criteria

Volume 13 is accepted when content, examples and architecture meet the master scope and the generated package passes structural, rendering, accessibility and integrity validation.

Validation	Acceptance
Scope	All 50 requested topics addressed
Examples	Production-oriented roots/modules/pipelines/policies/scripts included
State	Protection, locking, access and recovery defined
Services	Structured assessments completed for selected services
ADRs	Twelve full decisions included
Security	WIF/scoped identities/policy/state controls aligned to Volume 8/9
Operations	Drift, ownership, alerts and runbooks aligned to Volume 11
DR	Source/config/state recovery aligned to Volume 12
Rendering	DOCX/PDF visually clean
Integrity	Checksums and ZIP tests pass

60. Production Readiness Checklist

Each production root must pass the checklist before its first apply and after material module/provider/back-end changes.

Area	Check	Mandatory	Evidence
Repository	Protected branch, CODEOWNERS and signed/versioned releases	Yes	Repository settings
Root	Small lifecycle/trust boundary and named owner	Yes	Root catalog
State	Remote GCS backend, versioning, retention, access audit and recovery	Yes	Backend evidence
Identity	Keyless CI, separate plan/apply and no broad primitive roles	Yes	IAM export
Providers	Required version, provider constraints and committed lock file	Yes	Code/CI
Modules	Pinned version, docs, tests and compatibility policy	Yes	Module release
Plan	Saved plan, JSON, risk classification and independent approval	Yes	Release evidence
Policy	Static, security, terraform vet/OPA and post-apply controls	Yes	Gate reports
Apply	Scoped identity, lock timeout, logs and change record	Yes	Apply log
Drift	Scheduled refresh-only plan and response owner	Yes	Monitoring
Recovery	State/source/configuration recovery exercise	Tier based	Exercise record
FinOps	Labels, budgets and cost ownership enforced	Yes	Billing data
Operations	Runbook, escalation and vendor/provider support path	Yes	ORR

61. Cross-Volume Dependencies

Volume 13 implements infrastructure interfaces required by every prior volume and closes the final missing implementation layer.

Volume	Terraform dependency
01	Architecture principles, capability model and global decisions
02	Organization, project, network and landing-zone modules
03-06	Data platform, pipelines, mesh and analytics modules
07	AI foundation and platform modules
08	Identity, perimeter, KMS, logging and security modules
09	Source, CI, policy, approval and release pipeline
10	Labels, budgets, reservations and cost controls
11	Monitoring, SLO, alert and operational readiness modules
12	State/source/config recovery and DR tests
14	Industry composition using golden and domain modules

Phase 3 dependency. With Volume 13 complete, all 14 volumes are available. The next required series activity is the master prompt Phase 3 cross-volume architecture assurance.

62. Decisions and Next Actions

Volume 13 establishes the IaC authority and enables formal series closure after assurance.

Action	Owner	Priority
Approve ADR-1301 through ADR-1312	Enterprise Architecture Review Board	Immediate
Bootstrap secured state projects/buckets and recovery roles	Cloud Foundation/Security	Wave 1
Publish project, network, security, data and domain golden modules	Platform teams	Wave 1-3
Pilot plan/apply identities and policy gates	DevSecOps	Wave 1
Inventory/import priority unmanaged resources	Cloud Governance	Wave 2-5
Run Tier 0/1 state recovery exercise	SRE/BCDR	Before production
Begin Phase 3 cross-volume assurance	Series architecture owner	After package acceptance

63. References

Official Google Cloud and HashiCorp documentation was reviewed for the July 2026 baseline. Provider versions, resource schemas, service lifecycle, quotas, locations, IAM and security support must be revalidated during implementation.

Reference	Location
Terraform on Google Cloud	https://docs.cloud.google.com/docs/terraform
General style and structure best practices	https://docs.cloud.google.com/docs/terraform/best-practices/general-style-structure
Reusable module best practices	https://docs.cloud.google.com/docs/terraform/best-practices/reusable-modules
Terraform operations best practices	https://docs.cloud.google.com/docs/terraform/best-practices/operations
Terraform maturity model	https://docs.cloud.google.com/docs/terraform/maturity
Store Terraform state in Cloud Storage	https://docs.cloud.google.com/docs/terraform/resource-management/store-state
Export Google Cloud resources to Terraform	https://docs.cloud.google.com/docs/terraform/resource-management/export
Import Google Cloud resources into state	https://docs.cloud.google.com/docs/terraform/resource-management/import
Policy validation	https://docs.cloud.google.com/docs/terraform/policy-validation
Validate policies	https://docs.cloud.google.com/docs/terraform/policy-validation/validate-policies
Cloud Build with Terraform	https://docs.cloud.google.com/build/docs/terraform
Managing IaC with Terraform and Cloud Build	https://docs.cloud.google.com/docs/terraform/resource-management/managing-infrastructure-as-code
Infrastructure Manager overview	https://docs.cloud.google.com/infrastructure-manager/docs/overview
Terraform and Infrastructure Manager	https://docs.cloud.google.com/infrastructure-manager/docs/terraform
Terraform GCS backend	https://developer.hashicorp.com/terraform/language/backend/gcs
Terraform state locking	https://developer.hashicorp.com/terraform/language/state/locking
Terraform state	https://developer.hashicorp.com/terraform/language/state

Master scope source: Enterprise Google Cloud Data Platform Blueprint Series.pdf, Volume 13 and series-wide quality requirements. The master prompt is authoritative for numbering and Phase 3 assurance.